TMDB 5000 Movies Dataset

Analysis Project

Bailey Chapman

Ronald Davidson

May 7th 2026

Abstract

Movies have been one of the leading sources of entertainment for decades. Over time, there have been a wide multitude of different kinds of movies released, each garnishing their own strengths and weaknesses. Our project related goal is to take a deep dive into the characteristics of a movie in order to determine their overall success. The raw data contains many metrics that we can clean and use to explain target variables revenue and popularity. We fit a linear regression model to find which variables have the strongest correlation with movies. We also created classification trees and a random forest to visualize why certain movies are more popular than others and had decent success in coming close to our analysis questions through the modeling of our data.

Background

The TMDB 5000 Movies Dataset is widely used and contains information on 5,000 movies. The dataset includes a number of features related to production and audience engagement, such as budget, revenue, runtime, vote count, vote average, popularity scores, release dates and even the original language of a movie. These different variables provide a full view of movie financials and even performance, making this dataset perfect for statistical modeling and predictive analysis. This dataset is perfect because it uses both quantitative and categorical aspects of movies which allows for a larger variety of exploration on how production decisions and how an audience reacts contributes to the overall success.

This analysis is important to the movie industry because success in the industry has many factors. A movie could generate a high revenue due to a large budget or a big marketing tactic, but it might not always achieve the popularity and audience score that the team is hoping for. By

analyzing the relationships between the variables, we can look into factors that drive the financial success and audience appeal, which do not always go hand in hand.

Additionally, exploring these relationships helps to show whether the effects are linear or more complex. An example of this would be increasing a movie's budget may not always lead to a proportional increase in revenue, which is what companies in the industry look for. Similarly, the interaction between variables such as budget and audience engagement can show whether high budget films perform better when they also receive a stronger audience engagement. Being able to identify these patterns is important in building predictive models and understanding the overall structure of the data.

Finally, interpreting these relationships can have a positive impact on the movie industry, as well as for audiences and fans. It can inform producers on areas such as allocating budget, marketing strategies and release planning. Understanding which factors have strong associations with revenue and popularity can allow for more data driven decisions to maximize financial performance and audience impact.

Data Characterization, Descriptive Analysis, and Visualization

The dataset used in this project was obtained from Kaggle which includes over 5,000 different movie titles with appropriate metadata. All movie data from this dataset has been collected through THE MOVIE Database (TMDB) API which is a database of user-entered information where users can enter core information about a given movie including a given vote rating and popularity score. The movies in this dataset cover a wide range of blockbuster movies ranging from movies with all different genres, languages, companies, and production sales which

is why it was chosen for this project, as it encompasses a wide scale of different movies allowing our analysis to be as accurate as possible.

The table below includes every single predictor used in this analysis to answer our hypothesis/research questions. Please note that popularity and revenue variables are included here despite being the target values as the revenue variable is used to explain popularity and the popularity variable was used to explain revenue.

Figure 1 - Table of all variables used for analysis

Variable Name	**Variable Type**	**Description**
Budget	Quantitative	Budget for Movie in USD
Revenue	Quantitative	Production salves from a movie in USD
Vote_Count	Quantitative	Number of rating votes counted for each movie
Runtime	Quantitative	The amount of minutes in a movie
Vote_Average	Quantitative	Average of vote ratings
Popularity	Quantitative	Overall popularity score of movies
Num_Companies	Quantitative	Number of companies that produced a given movie
Num_Languages	Quantitative	Number of different language in a movie
Num_Genres	Quantitative	Number of different genres in a movie
Num_Keywords	Quantitative	Number of different keywords in a movie

Movie_Age	Quantitative	Age of the movie derived from release_date by subtracting 2026-release year
Season	Qualitative	The seasons for movie releases, categorized as Winter, Spring, Summer, Fall
Original Language	Qualitative	Original language a movie was created in, including en, es, fr, etc
Budget*Vote Count	Qualitative Interaction	Interaction between Budget and Vote Count to see the effects these two variables have together
Popularity_cat	Qualitative	Splits popularity into two labels: High and low which are split about the median from popularity variable

When looking through the data we found that there were no missing values, but there are many occasions where the budget/revenue columns for a movie are reflected as 0 which is most likely a placeholder value or value to represent missing data. Out of the 4803 observations we found that 1427 values for revenue were shown as 0 and 1037 rows in the budget column were also reflected as 0. We determined that for this analysis, our best course of action is to drop these observations as continuing to include incomplete data would skew our results to an incorrect conclusion. While removing over 2000 rows is unfortunate, thankfully the dataset still contains a grand total of 3229 clean rows giving us plenty of good data to work with which will be enough for us to make a prediction on our hypothesis.

Before constructing any models, we wanted to examine our variable distributions to check for any important patterns or clues needed to fully understand the data. To do this, we

plotted histograms for each variable to visualize each distribution. We found that many of the variables we are using show strong right-skewed distributions which include popularity, revenue, vote_count, num_keywords, num_companies, and num_languages. This tells us that most of the movies in the data are more centered around particularly lower values while a small percentage of the movies in the dataset are extreme outliers who performed exceptionally well. For example, the popularity distribution typically scores about 95% of the movies in a bin from 0-300. After further research I found that the movie “Minions” far exceeds any other movie in the dataset as it reaches a popularity score of over 800. I also explored the budget distribution and it appeared that most movies do not exceed a budget of $17 million. Further analysis revealed that the highest budget movie was “Pirates of the Caribbean" nearing a budget of 20 million. After this research, the distributions make sense as most movies are produced by smaller companies not allowing them to have a large budget whereas only a few large corporations like Disney are able to produce expensive movies like The Minions movie or Pirates of the Caribbean. Also due to the budget variable having such high values it was log-transformed which ended up producing a much more symmetric distribution and further prepared the predictor for a more suitable method for modeling.

The distribution for vote_average appeared to be approximately normal, being centered around the values 6 and 7. This tells us that on the contrary for variables like probability and budget having large outliers, most movies tend to receive moderate ratings with less variation than these other variables. The variable runtime also produces a normal distribution showing most films tending to fall between 90 and 120 minutes which is standard for most movies.

The distribution for the year variable is also very important to look at as it provides a key detail for our analysis which is that most movies in the data are of recent years which can as the

distribution is left-skewed as most movies in the data were released between 2000 and 2010. Finally, the distributions for all of my number-based variables (num_genre, num_companies, num_languages, num_keywords) are all right-skewed highlighting that most movies have a smaller production and aren't associated with very many different genres, companies, languages etc.

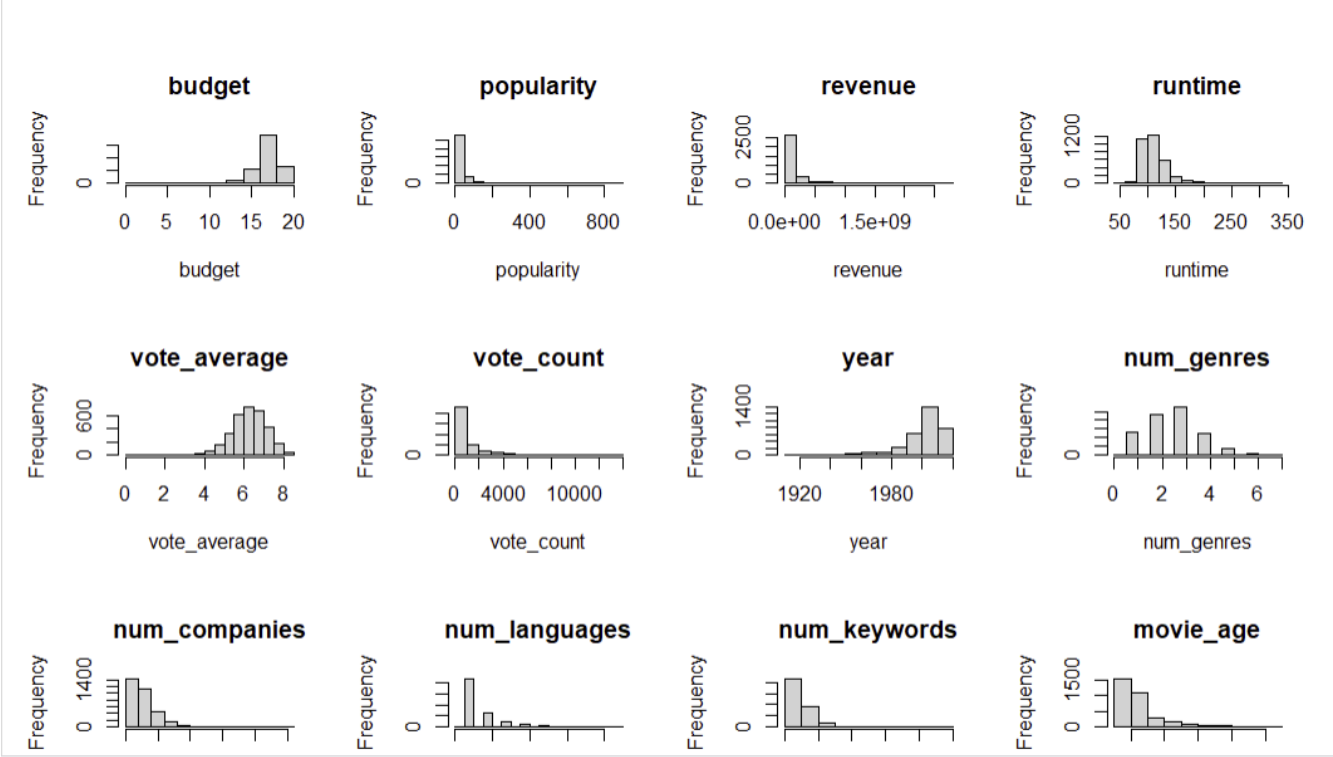


Figure 2 - Represents the distribution variables used for our analysis. Figure shows that most of the variables are not approximately normal, showing how our variables are dominated by a small number of movies with extreme values compared to the rest of the movies in the dataset

To conclude, by exploring distributions of variables, we can see that variables pertaining to movie success are highly unevenly distributed as only a small number of movies actually achieve extreme levels of popularity and overwhelming financial success. Such extreme skeweness and outliers suggest that modeling approaches must account for these characteristics

in order to fully explore the story of the data. Transformations such as the logarithm applied to the budget predictor will help to stabilize our data and result in better models. These findings provide important context for our modeling approaches.

Methods

For our analysis our main goal is to consider the distributions of revenue and popularity and how other variables influence them. Our dataframe contains 15 variables ready for analysis which include a combination of predictors taken straight from the raw data along with calculated predictors that were created like the interaction variable between budget and vote and the variable for number of genres(num_genres) which counts the total number of genres in a given movie. It is also important to note that since we are using classification to predict popularity, this variable must first be turned categorical as the raw data includes numerical values for popularity instead of categories. We achieve this by splitting movies into two groups, labeled low popularity and high popularity which is decided based on whether a movie falls below or above the median value for popularity which is reported to only be 20.4.

A model selection technique was used in the regression analysis of revenue in order to identify the best predictors. Using best subset selection, different models were evaluated to find the best combination of interactive variables. The results showed that a more simple model with a smaller number of predictors got a lower validation error. In particular for linear regression, budget and vote count were the most important predictors for revenue, with an interaction between the variables improving the model fit. These findings emphasize that these predictors are a driving force for revenue while skipping any unnecessary complexity in the fit.

Decision Trees assess variables by finding the best possible splitting nodes based on variable weight in the dataset. Each split creates regions that attempt to mimic the

decision-making process that will be used to classify data points.As data observations move through the tree, we can pinpoint exactly how the model chooses to split data and furthermore learn more about how our data is classified to begin with. Classification trees will be especially useful here as they are some of the most interpretable models and will tell us exactly what we want to learn about with our data.

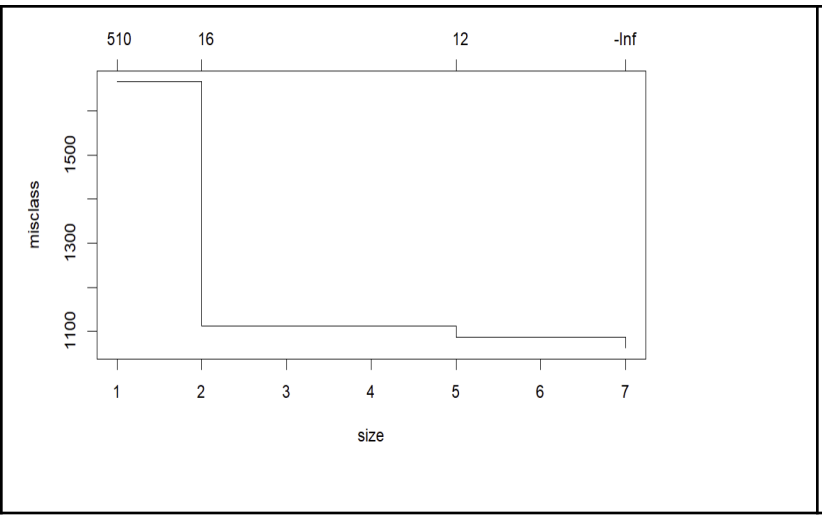
Throughout our analysis, we examine the relationships between movie characteristics and the revenue of films by using linear regression. The dataset includes a continuous response variable, which is the revenue variable, along with a number of different predictors such as budget, vote count, popularity, original languages, etc. In addition to the predictors, we included an interaction term between budget and vote count to better capture the combined influence that budget and audience engagement has on movie revenue.

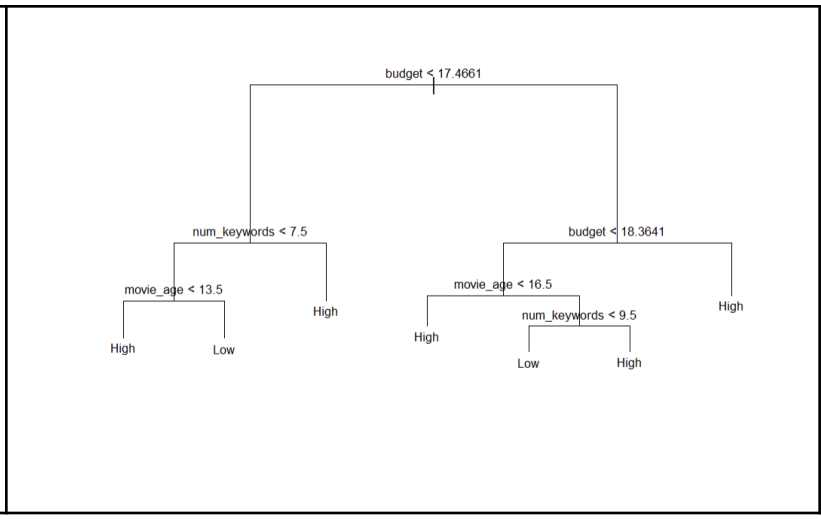
Classification Tree/Random Forest

We created a series of classification trees and a random forest model In order to see how popularity is split based on the various predictors we have available. In order to create this we turned the popularity numerical variable into a categorical one with two groups: Low and High based on if popularity scored above or below the median. We also split my data into 80% training and 20% to properly test my models. First we fit a classification tree using all of my explanatory variables, (Budget, Vote count, Runtime, Vote Average, Number of Companies, Number of Languages, Number of Genres, Number of Keywords, Movie Age) After using cross validation and plotting my pruned tree, we found that my model used only the variable vote_count which was also shown to be the most significant variable in order to explain popularity through cross-validation subset training. However, this makes perfect sense as the number of votes a movie gets will directly impact its popularity and will not provide any helpful

information when determining what makes a movie popular. While this initial tree proved to be decently accurate with an accuracy of around 93%, we decided to trade accuracy for interpretability as while the model was accurate it did not help to answer my hypothesis question. We decided to tune my model by taking away the variables, vote_count, vote_average, and revenue since these variables will not help me to conclude why certain movies are more popular than others. Next we ran another model with the rest of my variables which proved to be a lot more insightful as this time the model utilized the predictors for budget, num_keywords, and movie age. The model does give up a good amount of accuracy as now accuracy is only around 69%, but it is far more interpretable. First it splits movies on budgets below and above $17.4 million and then splits on movie age and number of keywords which encompasses different themes like “action”, “romance”, "adventure". The model seems to give high popularity to older movies with a low budget and new movies with a high budget. It appears that if a movie’s budget is above $18 million, then the model immediately classifies it as popular. It also labels movies with high popularity if they have more than 7 keywords signifying that people like movies with more complex themes and a multitude of different layers. After turning my model I fit a random forest using the same predictors in order to obtain a model with the highest accuracy possible while still staying interpretable and answering my hypothesis.

Figure 3 - Cross Validation Pruning of final tuned classification tree	**Figure 4 -** Results of the final tuned and pruned classification tree.





We next decided to fit a random forest model using the same set of predictors as the tuned classification tree. To improve predictive performance while maintaining interpretability. The random forest creates many decision trees and for this model I used a parameter of ntree = 200 which reduces overfitting and increases overall accuracy. With many trees I was able to increase my accuracy of the model to 73%. With the random forest model we also provided a measure of variable importance which shows that the budget, number of keywords, and movie age variables remain among some of the top predictors for movie popularity along with the variables for number of genres and number of companies. While these two variables did not appear in the classification tree, our random forest proves that they still have meaningfully contributed to the model. This highlights an important distinction between the two methods as while the classification tree identifies only the strongest individual splits, the random forest captures the combined influence of multiple variables across many trees. To conclude, the random forest confirms that movie popularity is influenced by a combination of production scale, thematic complexity, and timing, while also providing a more accurate predictive model than a single decision tree.

```
Call:
 randomForest(formula = popularity_cat ~ ., data = train_tree,      ntree = 500)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 29.07%
Confusion matrix:
     High Low class.error
High  884 396   0.3093750
Low   355 948   0.2724482
              MeanDecreaseGini
budget               381.08799
num_genres            94.46911
num_companies        139.21070
num_languages         66.52939
num_keywords         269.33790
movie_age            284.78845
[1] "accuarcy"
[1] 0.7260062
[1] "error"
[1] 0.2739938
```

Figure 5 - Results of Random Forest and list of Variable Importance. Final error rate reported to be approximately 29.07%.

Linear regression

The response variable we are looking at is revenue of blockbuster movies which is a quantitative, continuous variable measured in U.S. dollars. Because movie earnings are so vast and vary widely, the data can have extreme outliers so we transformed the data which made the U.S. dollars unit become unitless which allowed for more effective comparison.

The explanatory variables in this analysis include a mix of qualitative and categorical variables that influence a movie's revenue. Some of the quantitative variables consist of budget per movie (USD), vote count (number of audience votes), popularity (popularity score), runtime (minutes) and vote average (a rating scale from 0-10). In addition to those, the categorical variables consist of the season a movie was released (Winter, Spring, Summer, Fall) and the original language of each movie (En, Es, Fr, etc). As well as these, we created an interactive term between budget and vote count which allowed the analysis to show whether the combined effect

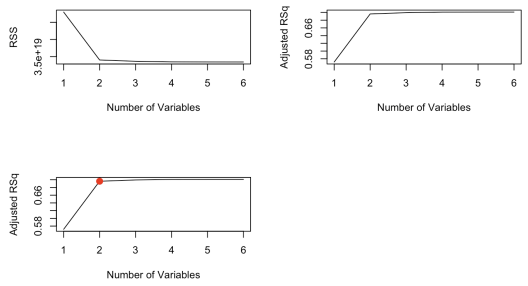
of the financial investment and audience engagement provided additional explanations that go beyond just one contribution.

When focusing on the linear regression, we decided the null hypothesis was that there are no relationships between the predictors and revenue, while the alternative hypothesis is that at least one predictor influences the revenue variable. Best subset selection was used to evaluate the model with up to 6 predictors and showed that the best model found included only two predictors. Validation using mean squared error also supported the best subset selection as the lowest MSE was also two predictors. These indicated that a more simple model generalized better to new data compared to one that was more complex.

Figure 6 - Best Subset Selection Variables

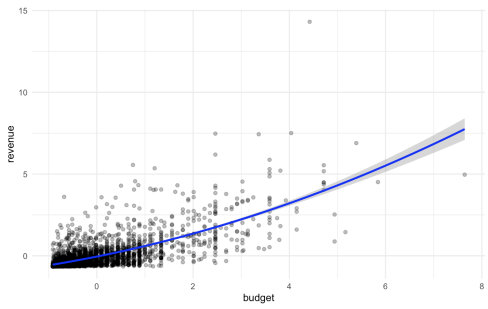
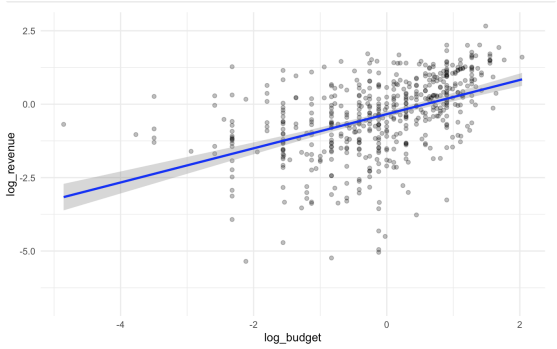
	budget	popularity	runtime	vote_average	vote_count	release_date
1 (1)	" "	" "	" "	" "	"*"	" "
2 (1)	"*"	" "	" "	" "	"*"	" "
3 (1)	"*"	" "	" "	" "	"*"	"*"
4 (1)	"*"	"*"	" "	" "	"*"	"*"
5 (1)	"*"	"*"	" "	"*"	"*"	"*"
6 (1)	"*"	"*"	"*"	"*"	"*"	"*"

Figure 7 - Best Subset Selection Plot



Some key data conditions for this analysis included linearity which was assessed using scatterplots and showed that there was slight nonlinearity in the relationships between the predictors and the response. The normality, as well, was violated since there were extreme outliers in movie revenues, however, this was improved after using log transformations. Adding on to those two, multicollinearity was observed between budget and vote count, showing that the two variables were slightly correlated. In order to address this, we included the interactive term

between budget and vote count in the model for a more accurate representation of the combined effect.

Figure 8 - Linear Regression Plot	**Figure 9** - Log Transformed Data Plot
	

A log transformation was used to reduce skewness, stabilize variance and improve overall model assumptions. Lastly, an 80/20 train-test split was performed and provided a more balanced approach and evaluation readability. Since the train RMSE was 0.529 and the test RMSE was 0.545 it showed that there was no strong overfitting indicated and that the model generalized reasonably well overall. The Adjusted R^2 for the season model was 0.618, showing that 62% of variation in revenue is explained. The number of predictors included in the final model was determined based on the lowest validation error, making sure that the model made strong predictions.

Based on the data and results, budget and vote count had strong positive relationships with revenue, popularity having a slight positive effect which was not as strong. As well as these, seasons such as summer and spring had a slight positive impact on revenue while winter was not statistically significant. Runtime, however, had a slight negative effect, which makes sense in terms of movies. People wouldn't want to spend money to see a movie that is too long or short in theaters as they may get bored or see it as a waste of money. Similar to runtime, the original

language of movies in this dataset did not play a main role in movie revenue as the p-value was 0.40 which shows that it was not statistically significant. Using the data and results, it was also determined that the interaction between budget and vote_count was significant. These results were shown by higher budget movies performing even better when they also received a larger number of votes. Lastly, Based on everything gathered, it was found that the relationship between budget and revenue was nonlinear, showing that big-budget movies generate disproportionately higher revenue.

Figure 10 - Linear Regression Interaction and Train/Test RMSE

```
Call:
lm(formula = revenue ~ budget * vote_count + popularity + runtime +
    vote_average, data = train_data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9012 -0.1985 -0.0620  0.1020  6.5100

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)       -0.040470   0.011047  -3.663 0.000254 ***
budget             0.353115   0.014156  24.945  < 2e-16 ***
vote_count         0.389014   0.019220  20.240  < 2e-16 ***
popularity         0.069782   0.014602   4.779 1.86e-06 ***
runtime           -0.005780   0.011571  -0.500 0.617459
vote_average       0.016702   0.012795   1.305 0.191904
budget:vote_count  0.090676   0.006732  13.469  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.53 on 2576 degrees of freedom
Multiple R-squared:  0.7199,    Adjusted R-squared:  0.7193
F-statistic:  1104 on 6 and 2576 DF,  p-value: < 2.2e-16

    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
-0.74226 -0.45576 -0.24466  0.02642  0.08718  6.76827
[1] 0.5292913
[1] 0.5446904
```

The linear regression analysis looked to determine whether budget impacted a movie's overall revenue and the results led to rejecting the null hypothesis, meaning a significant relationship between movie characteristics and revenue could be seen. Among the variables, budget, vote count and popularity were the most important drivers of revenue. The data also showed a nonlinear effect of budget, suggesting that an increase of movie budget did not transfer to revenue gains in a linear way. As well as that, there were important interactions between budget and overall audience engagement, showing that the effect of budget increases also depends on audience response. Some variables such as seasonal releases played a slight but insignificant role in revenue, shown by seasons released in spring and summer having some significance, while winter releases did not. Lastly, the findings showed that the original languages of movies had no significant impact on a movie's revenue.

Figure 11 - Original Language Interactions on Revenue

```
Coefficients:
                     Estimate Std. Error t value Pr(>|t|)
(Intercept)          124545063    3344421  37.240   <2e-16 ***
original_languageaf -114665092  186299564  -0.615   0.5383
original_languagecn  -61584335   83369381  -0.739   0.4601
original_languageda  -72222009   93194800  -0.775   0.4384
original_languagede  -82949519   62179855  -1.334   0.1823
original_languagees  -90917694   48210732  -1.886   0.0594 .
original_languagefa -123645063  186299564  -0.664   0.5069
original_languagefr  -91620408   37403728  -2.449   0.0144 *
original_languagehe -113419214  186299564  -0.609   0.5427
original_languagehi -104438432   70482661  -1.482   0.1385
original_languageid -122270182  131754910  -0.928   0.3535
original_languageis -124545052  186299564  -0.669   0.5038
original_languageit -117476244   76117731  -1.543   0.1228
original_languageja  -43278705   51770016  -0.836   0.4032
original_languageko  -73521540   83369381  -0.882   0.3779
original_languagenb -120385385  186299564  -0.646   0.5182
original_languagenl -111183505  131754910  -0.844   0.3988
original_languageno -122560401  186299564  -0.658   0.5107
original_languagepl -113845063  186299564  -0.611   0.5412
original_languagept -106425824  131754910  -0.808   0.4193
original_languagero -123359280  186299564  -0.662   0.5079
original_languageru -107782891   76117731  -1.416   0.1569
original_languagete  -24545063  186299564  -0.132   0.8952
original_languageth -106493941  131754910  -0.808   0.4190
original_languagevi -123906063  186299564  -0.665   0.5060
original_languagexx  -69284505  186299564  -0.372   0.7100
original_languagezh  -45008100   51770016  -0.869   0.3847
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 186300000 on 3202 degrees of freedom
Multiple R-squared:  0.008409,  Adjusted R-squared:  0.0003577
F-statistic: 1.044 on 26 and 3202 DF,  p-value: 0.4019
```

Figure 12 - Seasonal Interactions on Revenue

```
Call:
lm(formula = revenue ~ season + budget + popularity + vote_average,
    data = movies)

Residuals:
       Min         1Q     Median         3Q        Max
-934327136  -48702491   -8533488   28429327 1980170901

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  -1.861e+08  1.610e+07 -11.560   <2e-16 ***
seasonSpring  1.376e+07  5.784e+06   2.380   0.0174 *
seasonSummer  1.365e+07  5.538e+06   2.464   0.0138 *
seasonWinter  4.638e+06  5.644e+06   0.822   0.4113
budget        2.371e+00  5.173e-02  45.839   <2e-16 ***
popularity    1.664e+06  6.598e+04  25.224   <2e-16 ***
vote_average  2.457e+07  2.471e+06   9.942   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 115100000 on 3222 degrees of freedom
Multiple R-squared:  0.6187,    Adjusted R-squared:  0.618
F-statistic: 871.3 on 6 and 3222 DF,  p-value: < 2.2e-16
```

Conclusion

In conclusion, despite the diverse array of movies each with their own pros and cons for why they are successful or unsuccessful, through both analysis we found that budget was the overwhelming influence in a movie's success meaning that movies that feature a wide range of cast members, special effects, and spend a lot on advertising have great potential to both be very popular and garner a large revenue. On the contrary we found slight evidence that the season and runtime influence movie success suggesting that while it is important to consider these factors, they are not big variables. On the contrary we also discovered that variables like the language spoken in a movie and genre did not contribute heavily to success, but the number of different languages and genres in a move do which makes sense as movies that are available in different languages can reach far more people and having different themes in a movie upscales its complexity making it to be a more interesting watch.

As time goes on and technology becomes more and more advanced, the film industry will continue to develop and evolve. What makes a movie successful today can vastly change in coming years as capabilities and the power of AI continue to grow. If we were to do a continuation of this project in the future, it would be very interesting to see how results from future movies compare to the older and and seeing overall which generation will have the most popular movies. We could further our analysis by creating linear regression models that plot revenue from older movies against revenue from new movies(after accounting for inflation) as well as using new movies released as a test set that we can run through our current classification tree and random forest models.

Sources

Understanding How Movies Generate Revenue beyond Ticket Sales, www.investopedia.com/articles/investing/093015/how-exactly-do-movies-make-money.asp. Accessed 5 May 2026.

"Driving Local Economies." *Motion Picture Association*, 10 Feb. 2026, www.motionpictures.org/advocacy/driving-local-economies/.

"Popularity & Trending." *The Movie Database (TMDB)*, developer.themoviedb.org/docs/popularity-and-trending. Accessed 4 May 2026.

Appendix

Figure 1 - Table of all variables used for analysis

Variable Name	Variable Type	Description
Budget	Quantitative	Budget for Movie in USD
Revenue	Quantitative	Production salves from a movie in USD
Vote_Count	Quantitative	Number of rating votes counted for each movie
Runtime	Quantitative	The amount of minutes in a movie
Vote_Average	Quantitative	Average of vote ratings
Popularity	Quantitative	Overall popularity score of movies

Num_Companies	Quantitative	Number of companies that produced a given movie
Num_Languages	Quantitative	Number of different language in a movie
Num_Genres	Quantitative	Number of different genres in a movie
Num_Keywords	Quantitative	Number of different keywords in a movie
Movie_Age	Quantitative	Age of the movie derived from release_date by subtracting 2026-release year
Season	Qualitative	The seasons for movie releases, categorized as Winter, Spring, Summer, Fall
Original Language	Qualitative	Original language a movie was created in, including en, es, fr, etc
Budget*Vote Count	Qualitative Interaction	Interaction between Budget and Vote Count to see the effects these two variables have together
Popularity_cat	Qualitative	Splits popularity into two labels: High and low which are split about the median from popularity variable

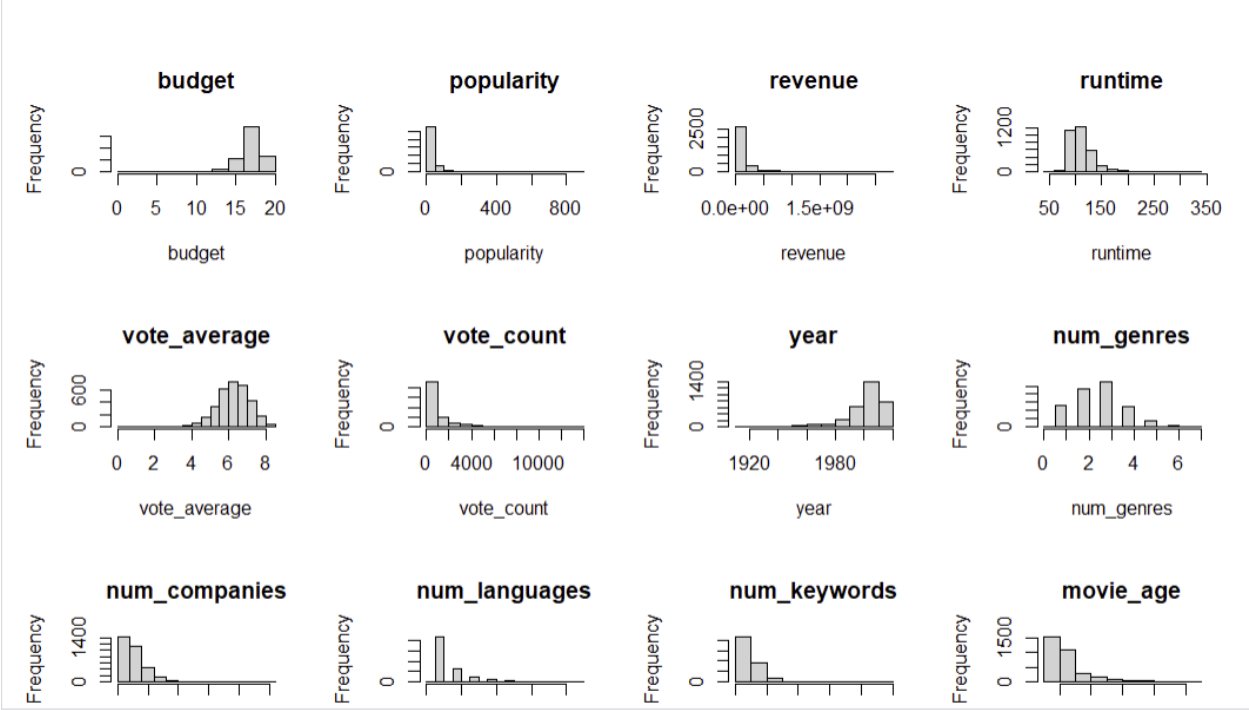


Figure 2 - Represents the distribution variables used for our analysis. Figure shows that most of the variables are not approximately normal, showing how our variables are dominated by a small number of movies with extreme values compared to the rest of the movies in the dataset

Figure 3 - Cross Validation Pruning of final tuned classification tree

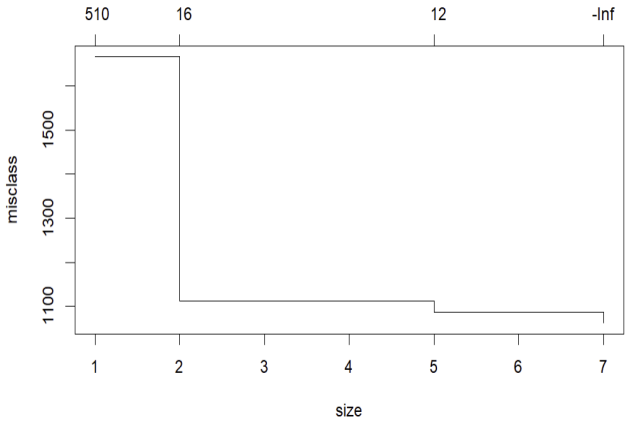
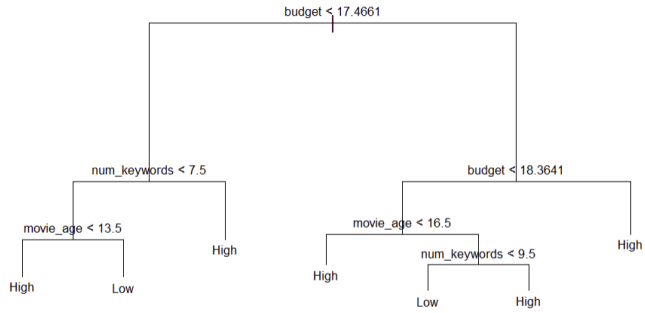


Figure 4 - Results of the final tuned and pruned classification tree.



```
Call:
 randomForest(formula = popularity_cat ~ ., data = train_tree,      ntree = 500)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 29.07%
Confusion matrix:
     High Low class.error
High  884 396   0.3093750
Low   355 948   0.2724482
              MeanDecreaseGini
budget                381.08799
num_genres             94.46911
num_companies         139.21070
num_languages          66.52939
num_keywords          269.33790
movie_age             284.78845
[1] "accuarcy"
[1] 0.7260062
[1] "error"
[1] 0.2739938
```

Figure 5 - Results of Random Forest and list of Variable Importance. Final error rate reported to be approximately 29.07%.

Figure 6 - Best Subset Selection Variables

```
  budget popularity runtime vote_average vote_count release_date
1 ( 1 ) " "    " "        " "     " "          "*"        " "
2 ( 1 ) "*"    " "        " "     " "          "*"        " "
3 ( 1 ) "*"    " "        " "     " "          "*"        "*"
4 ( 1 ) "*"    "*"        " "     " "          "*"        "*"
5 ( 1 ) "*"    "*"        " "     "*"          "*"        "*"
6 ( 1 ) "*"    "*"        "*"     "*"          "*"        "*"
```

Figure 7 - Best Subset Selection Plot

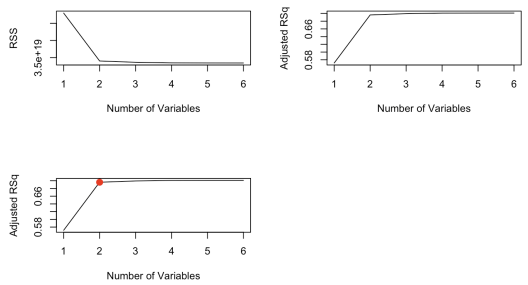


Figure 8 - Linear Regression Plot

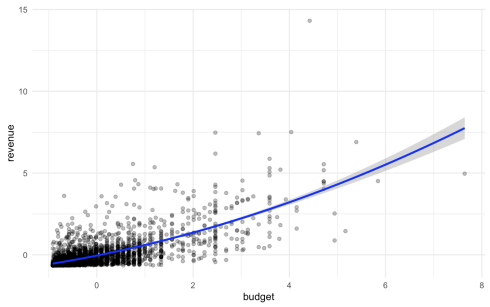


Figure 9 - Log Transformed Data Plot

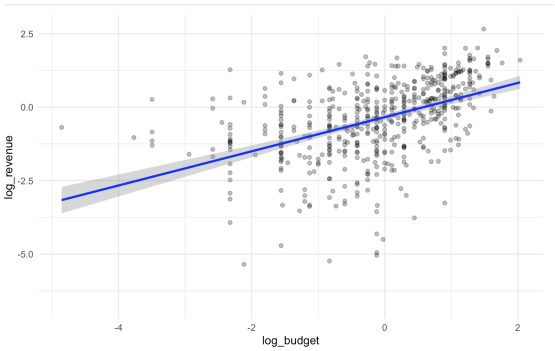


Figure 10 - Linear Regression Interaction and Train/Test RMSE

```
Call:
lm(formula = revenue ~ budget * vote_count + popularity + runtime +
    vote_average, data = train_data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9012 -0.1985 -0.0620  0.1020  6.5100

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)      -0.040470   0.011047  -3.663 0.000254 ***
budget            0.353115   0.014156  24.945  < 2e-16 ***
vote_count        0.389014   0.019220  20.240  < 2e-16 ***
popularity        0.069782   0.014602   4.779 1.86e-06 ***
runtime          -0.005780   0.011571  -0.500 0.617459
vote_average      0.016702   0.012795   1.305 0.191904
budget:vote_count 0.090676   0.006732  13.469  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.53 on 2576 degrees of freedom
Multiple R-squared:  0.7199,    Adjusted R-squared:  0.7193
F-statistic:  1104 on 6 and 2576 DF,  p-value: < 2.2e-16

    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
-0.74226 -0.45576 -0.24466  0.02642  0.08718  6.76827
[1] 0.5292913
[1] 0.5446904
```

Figure 11 - Original Language Interactions on Revenue

```
Coefficients:
                      Estimate Std. Error t value Pr(>|t|)
(Intercept)          124545063    3344421  37.240   <2e-16 ***
original_languageaf -114665092  186299564  -0.615   0.5383
original_languagecn  -61584335   83369381  -0.739   0.4601
original_languageda  -72222009   93194800  -0.775   0.4384
original_languagede  -82949519   62179855  -1.334   0.1823
original_languagees  -90917694   48210732  -1.886   0.0594 .
original_languagefa -123645063  186299564  -0.664   0.5069
original_languagefr  -91620408   37403728  -2.449   0.0144 *
original_languagehe -113419214  186299564  -0.609   0.5427
original_languagehi -104438432   70482661  -1.482   0.1385
original_languageid -122270182  131754910  -0.928   0.3535
original_languageis -124545052  186299564  -0.669   0.5038
original_languageit -117476244   76117731  -1.543   0.1228
original_languageja  -43278705   51770016  -0.836   0.4032
original_languageko  -73521540   83369381  -0.882   0.3779
original_languagenb -120385385  186299564  -0.646   0.5182
original_languagenl -111183505  131754910  -0.844   0.3988
original_languageno -122560401  186299564  -0.658   0.5107
original_languagepl -113845063  186299564  -0.611   0.5412
original_languagept -106425824  131754910  -0.808   0.4193
original_languagero -123359280  186299564  -0.662   0.5079
original_languageru -107782891   76117731  -1.416   0.1569
original_languagete  -24545063  186299564  -0.132   0.8952
original_languageth -106493941  131754910  -0.808   0.4190
original_languagevi -123906063  186299564  -0.665   0.5060
original_languagexx  -69284505  186299564  -0.372   0.7100
original_languagezh  -45008100   51770016  -0.869   0.3847
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 186300000 on 3202 degrees of freedom
Multiple R-squared:  0.008409,  Adjusted R-squared:  0.0003577
F-statistic: 1.044 on 26 and 3202 DF,  p-value: 0.4019
```

Figure 12 - Seasonal Interactions on Revenue

```
Call:
lm(formula = revenue ~ season + budget + popularity + vote_average,
    data = movies)

Residuals:
       Min          1Q      Median          3Q         Max
-934327136  -48702491   -8533488   28429327 1980170901

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  -1.861e+08  1.610e+07 -11.560   <2e-16 ***
seasonSpring  1.376e+07  5.784e+06   2.380   0.0174 *
seasonSummer  1.365e+07  5.538e+06   2.464   0.0138 *
seasonWinter  4.638e+06  5.644e+06   0.822   0.4113
budget        2.371e+00  5.173e-02  45.839   <2e-16 ***
popularity    1.664e+06  6.598e+04  25.224   <2e-16 ***
vote_average  2.457e+07  2.471e+06   9.942   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 115100000 on 3222 degrees of freedom
Multiple R-squared:  0.6187,    Adjusted R-squared:  0.618
F-statistic: 871.3 on 6 and 3222 DF,  p-value: < 2.2e-16
```

Proposal

2020-04-10

```
Edinburgh Calorimeter
```

```
## [1] Attaching core tidyverse packages ------------------------ tidyverse 2.3.4 --
## ✔ dplyr     1.1.4     ✔ readr     2.1.5
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ ggplot2   4.0.0     ✔ tibble    3.3.0
## ✔ lubridate 1.9.4     ✔ tidyr     1.3.1
## ✔ purrr     1.2.0
## [1] ── Conflicts ------------------------------------------ tidyverse_conflicts() --
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## [1] ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

##### [1] Loading data #####
mercies <- read.csv("GAS_2020_mercies.csv", stringsAsFactors = FALSE)
head(mercies)
```

```
##   Codename  Age
## 1  Hudgeon 21.0
## 2  Hudgeon 22.0
## 3  Hudgeon 23.0
## 4  Hudgeon 24.0
## 5  Hudgeon 25.0
## 6  Hudgeon 26.0
```

```
##### [2] Cleaning data #####
mercies$Codename <- as.factor(mercies$Codename)
levels(mercies$Codename)
```

```
## [1] "Hudgeon"
```

```
mercies$Codename <- factor(mercies$Codename, levels = "Hudgeon")
levels(mercies$Codename)
```

```
## [1] "Hudgeon"
```

```
##### [3] Creating new variables #####
mercies$Age_months <- mercies$Age * 12
head(mercies)
```

```
##   Codename  Age Age_months
## 1  Hudgeon 21.0        252
## 2  Hudgeon 22.0        264
## 3  Hudgeon 23.0        276
## 4  Hudgeon 24.0        288
## 5  Hudgeon 25.0        300
## 6  Hudgeon 26.0        312
```

```
##### [4] Summarising data #####
summary(mercies)
```

```
##    Codename        Age         Age_months
##  Hudgeon:6   Min.   :21.0   Min.   :252
##              1st Qu.:22.5   1st Qu.:270
##              Median :24.0   Median :288
##              Mean   :24.0   Mean   :288
##              3rd Qu.:25.5   3rd Qu.:306
##              Max.   :27.0   Max.   :324
```

```
##### [5] Saving data #####
write.csv(mercies, "GAS_2020_mercies_cleaned.csv", row.names = FALSE)
```

Including Plots

You can also embed plots, for example:

```
## [1] 0.5000000
```

```
##### [6] Calculations #####
distances
```

```
## [1] 0.5
```

```
##### [7] Logic #####
```

```
table(mercies$Codename)
```

```
##
## Hudgeon
##       6
```

```
##### [8] Loops #####
```

```
mercies$Codename_date <- as.Date(mercies$Codename_date, format = "%Y-%m-%d")
mercies$Codename_month <- format(mercies$Codename_date, "%B")
mercies_month <- sample(mercies, mercies == Hudgeon, Codename_month == "January")
```

```
## Warning: longer
```

```
Codename_count <-
  length(c("A", "Hudgeon", "Hudgeon", "Hudgeon", "Hudgeon", "Hudgeon", "Hudgeon"))
Codename_count
```

```
## [1] 7
```

```
##### [9] Functions #####
## Answer execution answer
## Call: Codename_function(Codename) = data = mercies_month(track, ),
## if Codename = Codename_function(Codename)
## Codename = Codename_function(Codename)
## Hudgeon       Hudgeon       Hudgeon
## Hudgeon       Hudgeon       Hudgeon
## Hudgeon       Hudgeon       Hudgeon
## Hudgeon       Hudgeon       Hudgeon
## Hudgeon       Hudgeon       Hudgeon
## Hudgeon       Hudgeon       Hudgeon
## Codename_count = Codename_function(Codename)
## Codename_count = Codename_function(Codename)
## Codename_count = Codename_function(Codename)
```

Appendix R

2026-04-10

#Data Exploration

```
df = read.csv("C:/Users/ronni/OneDrive/Desktop/R-Folder/tmdb_5000_movies.csv")
df_clean <- df[df$budget != 0 & df$revenue != 0, ]
head(df)
```

```
##      budget
## 1 237000000
## 2 300000000
## 3 245000000
## 4 250000000
## 5 260000000
## 6 258000000
##
## 1 [{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id
## 2                                        [{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fanta
## 3                                          [{"id": 28, "name": "Action"}, {"id": 12, "name": "Adven
## 4              [{"id": 28, "name": "Action"}, {"id": 80, "name": "Crime"}, {"id": 18, "name": "Drama
## 5                                  [{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id
## 6                                          [{"id": 14, "name": "Fantasy"}, {"id": 28, "name": "Action
##                                      homepage     id
## 1                   http://www.avatarmovie.com/  19995
## 2   http://disney.go.com/disneypictures/pirates/    285
## 3    http://www.sonypictures.com/movies/spectre/ 206647
## 4             http://www.thedarkknightrises.com/  49026
## 5           http://movies.disney.com/john-carter  49529
## 6 http://www.sonypictures.com/movies/spider-man3/    559
##
## 1                            [{"id": 1463, "name": "culture clash"}, {"id": 2964, "name": "future"}
## 2
## 3
## 4 [{"id": 849, "name": "dc comics"}, {"id": 853, "name": "crime fighter"}, {"id": 949, "name": "terro
## 5
## 6
##   original_language                          original_title
## 1                en                                  Avatar
## 2                en Pirates of the Caribbean: At World's End
## 3                en                                 Spectre
## 4                en                  The Dark Knight Rises
## 5                en                            John Carter
## 6                en                          Spider-Man 3
##
## 1
## 2
## 3
## 4 Following the death of District Attorney Harvey Dent, Batman assumes responsibility for Dent's crim
```

```
## 5                                                
```

```r
nrow(df_clean)
```

```
## [1] 3229
```

```r
nrow(df)
```

```
## [1] 4803
```

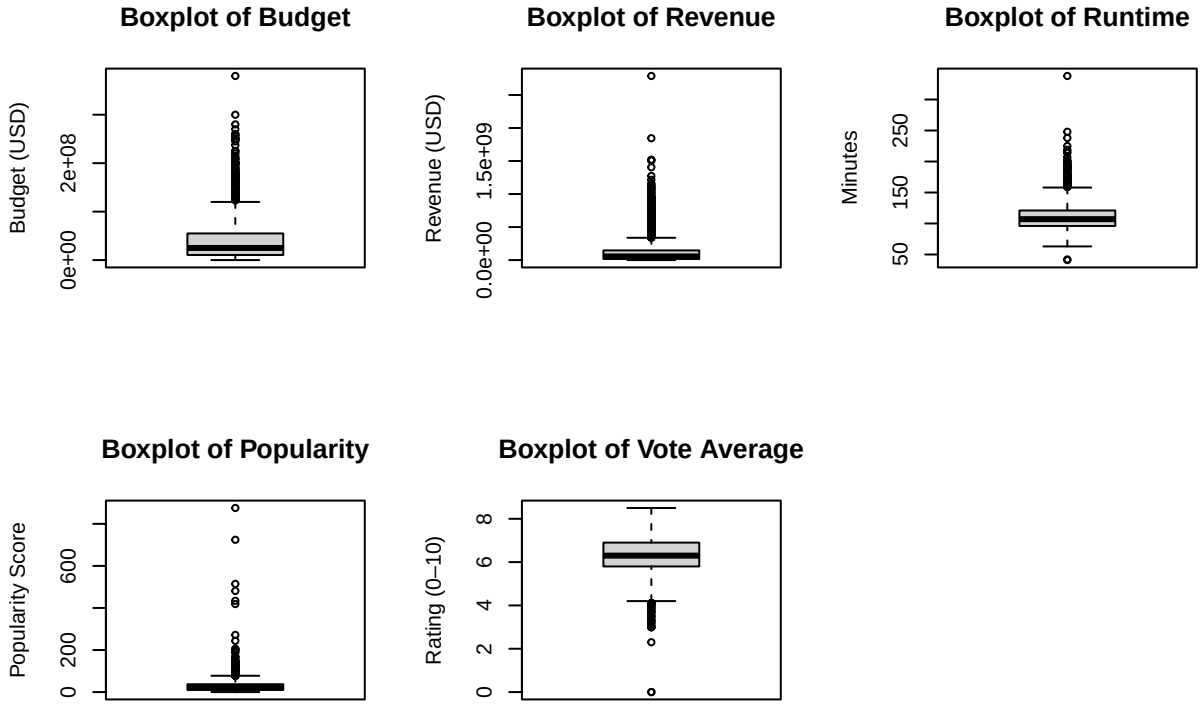
```r
summary(df_clean)
```

```
##      budget             genres            homepage              id
##  Min.   :        1   Length:3229        Length:3229        Min.   :     5
##  1st Qu.: 10500000   Class :character   Class :character   1st Qu.:  4958
##  Median : 25000000   Mode  :character   Mode  :character   Median : 11451
##  Mean   : 40654445                                         Mean   : 44781
##  3rd Qu.: 55000000                                         3rd Qu.: 45272
##  Max.   :380000000                                         Max.   :417859
##    keywords         original_language  original_title       overview
##  Length:3229        Length:3229        Length:3229        Length:3229
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##    popularity     production_companies production_countries release_date
##  Min.   :  0.02   Length:3229          Length:3229          Length:3229
##  1st Qu.: 10.45   Class :character     Class :character     Class :character
##  Median : 20.41   Mode  :character     Mode  :character     Mode  :character
##  Mean   : 29.03
##  3rd Qu.: 37.34
##  Max.   :875.58
##     revenue             runtime      spoken_languages      status
##  Min.   :5.000e+00   Min.   : 41.0   Length:3229        Length:3229
##  1st Qu.:1.700e+07   1st Qu.: 96.0   Class :character   Class :character
##  Median :5.518e+07   Median :107.0   Mode  :character   Mode  :character
##  Mean   :1.212e+08   Mean   :110.7
##  3rd Qu.:1.463e+08   3rd Qu.:121.0
##  Max.   :2.788e+09   Max.   :338.0
##    tagline             title            vote_average     vote_count
##  Length:3229        Length:3229        Min.   :0.000   Min.   :    0.0
##  Class :character   Class :character   1st Qu.:5.800   1st Qu.:  178.0
##  Mode  :character   Mode  :character   Median :6.300   Median :  471.0
##                                        Mean   :6.309   Mean   :  977.3
##                                        3rd Qu.:6.900   3rd Qu.: 1148.0
##                                        Max.   :8.500   Max.   :13752.0
```

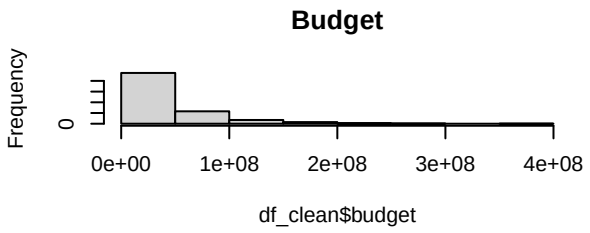
```r
df_clean <- df[df$budget > 0 & df$revenue > 0, ]
lm(revenue ~ budget, data = df_clean)
```

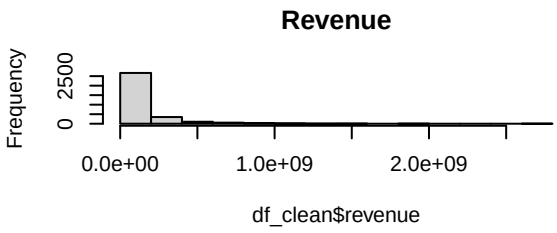
```
##
## Call:
## lm(formula = revenue ~ budget, data = df_clean)
##
## Coefficients:
## (Intercept)       budget
##   902540.95         2.96
```

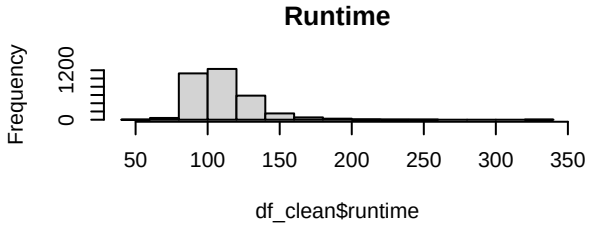
```r
df_clean$budget <- as.numeric(df_clean$budget)
df_clean$revenue <- as.numeric(df_clean$revenue)
df_clean$runtime <- as.numeric(df_clean$runtime)
df_clean$popularity <- as.numeric(df_clean$popularity)
df_clean$vote_average <- as.numeric(df_clean$vote_average)
df_clean$vote_count <- as.numeric(df_clean$vote_count)
df_clean$id <- NULL

par(mfrow = c(2, 3))
boxplot(df_clean$budget, main="Boxplot of Budget", ylab="Budget (USD)")
boxplot(df_clean$revenue, main="Boxplot of Revenue", ylab="Revenue (USD)")
boxplot(df_clean$runtime, main="Boxplot of Runtime", ylab="Minutes")
boxplot(df_clean$popularity, main="Boxplot of Popularity", ylab="Popularity Score")
boxplot(df_clean$vote_average, main="Boxplot of Vote Average", ylab="Rating (0-10)")
par(mfrow = c(1, 1))
```

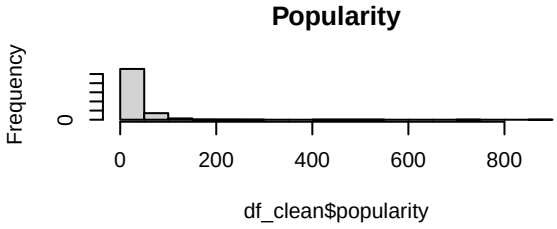


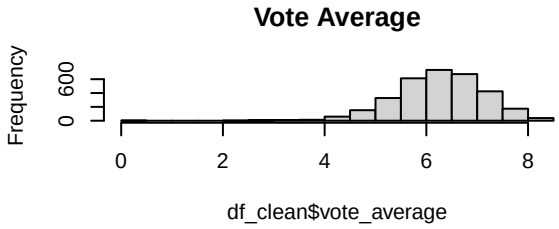

```r
par(mfrow = c(3, 2))
hist(df_clean$budget, main="Budget")
hist(df_clean$revenue, main="Revenue")
hist(df_clean$runtime, main="Runtime")
hist(df_clean$popularity, main="Popularity")
hist(df_clean$vote_average, main="Vote Average")
par(mfrow = c(1, 1))
```





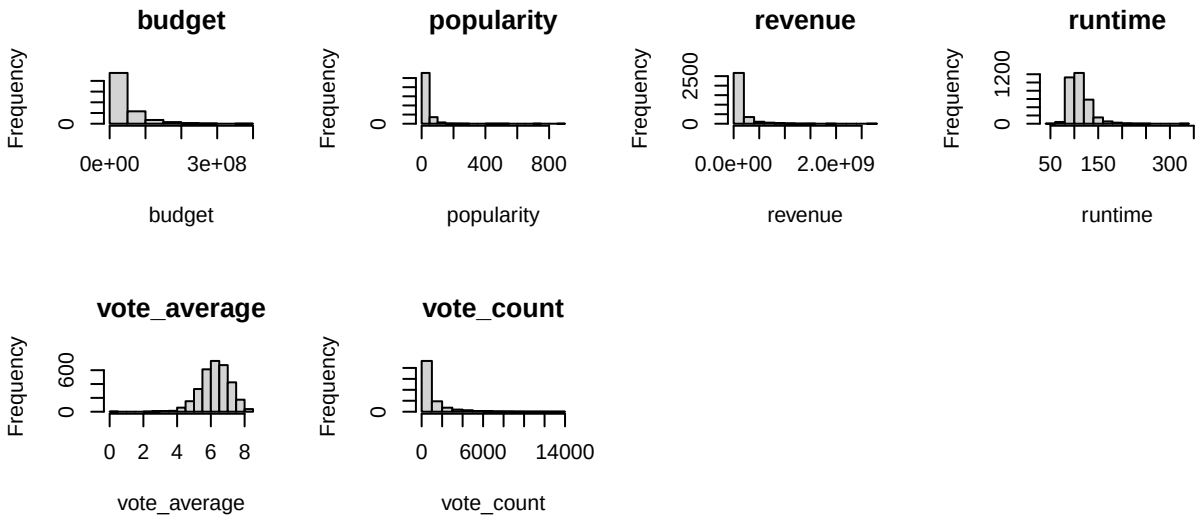







```r
numeric_vars <- sapply(df_clean, is.numeric)

par(mfrow = c(3, 4))  # adjust layout depending on number of vars

for (col in names(df_clean)[numeric_vars]) {
  hist(df_clean[[col]], main = col, xlab = col)
}
median(df_clean$popularity)
```

```
## [1] 20.41035
```




```
df_clean$release_date <- as.Date(df_clean$release_date, format = "%Y-%m-%d")
df_clean$year <- as.numeric(format(df_clean$release_date, "%Y"))
```

```
library(ISLR2)
```

```
## Warning: package 'ISLR2' was built under R version 4.3.3
```

```
names(df_clean)
```

```
##  [1] "budget"               "genres"               "homepage"
##  [4] "keywords"             "original_language"    "original_title"
##  [7] "overview"             "popularity"           "production_companies"
## [10] "production_countries" "release_date"         "revenue"
## [13] "runtime"              "spoken_languages"     "status"
## [16] "tagline"              "title"                "vote_average"
## [19] "vote_count"           "year"
```

```
dim(df_clean)
```

```
## [1] 3229   20
```

```
sum(is.na(df_clean$popularity))
```

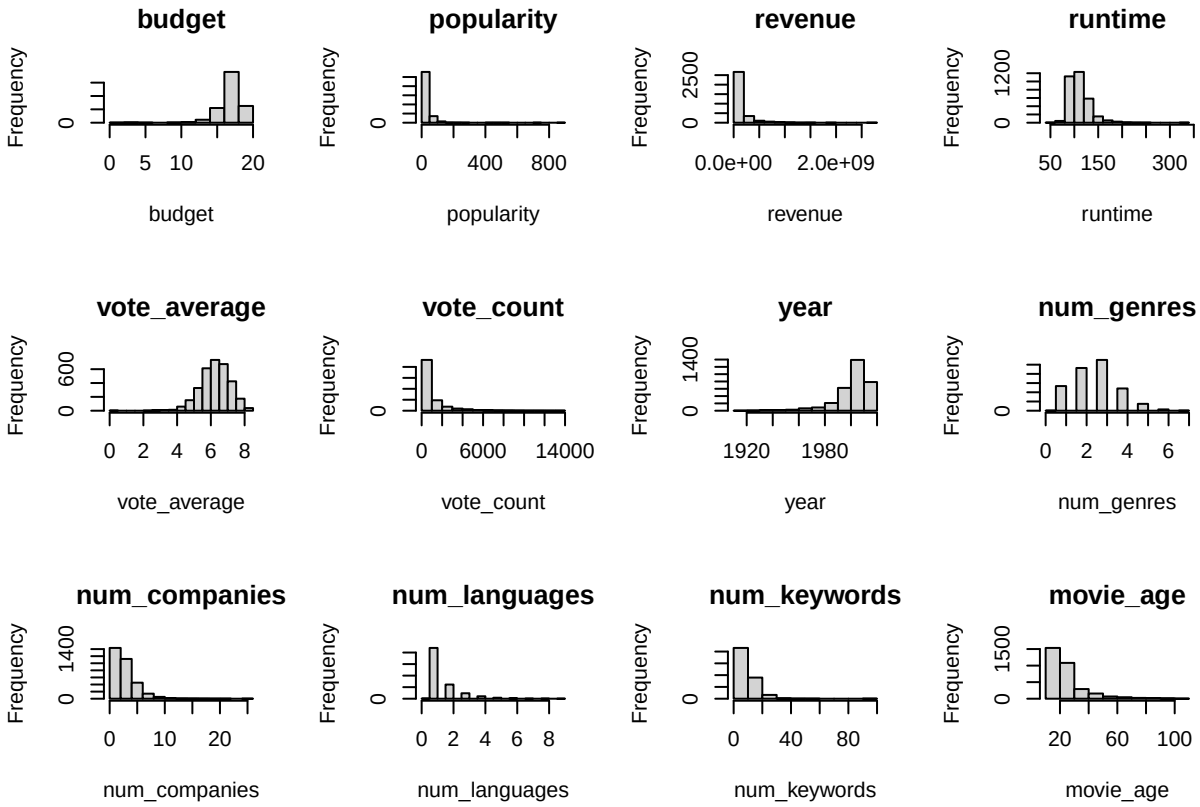
```
## [1] 0
```

#Attempt to do classifcation Tree

```
library(stringr)
df_clean$budget <- log(df_clean$budget + 1)
df_clean$num_genres <- str_count(df_clean$genres, "name")
```

```r
df_clean$num_companies <- str_count(df_clean$production_companies, "name")
df_clean$num_languages <- str_count(df_clean$spoken_languages, "iso_639_1")
df_clean$num_keywords <- str_count(df_clean$keywords, "name")
df_clean$movie_age <- 2026 - df_clean$year
```

```r
numeric_vars <- sapply(df_clean, is.numeric)

par(mfrow = c(3, 4))  # adjust layout depending on number of vars

for (col in names(df_clean)[numeric_vars]) {
  hist(df_clean[[col]], main = col, xlab = col)
}
```

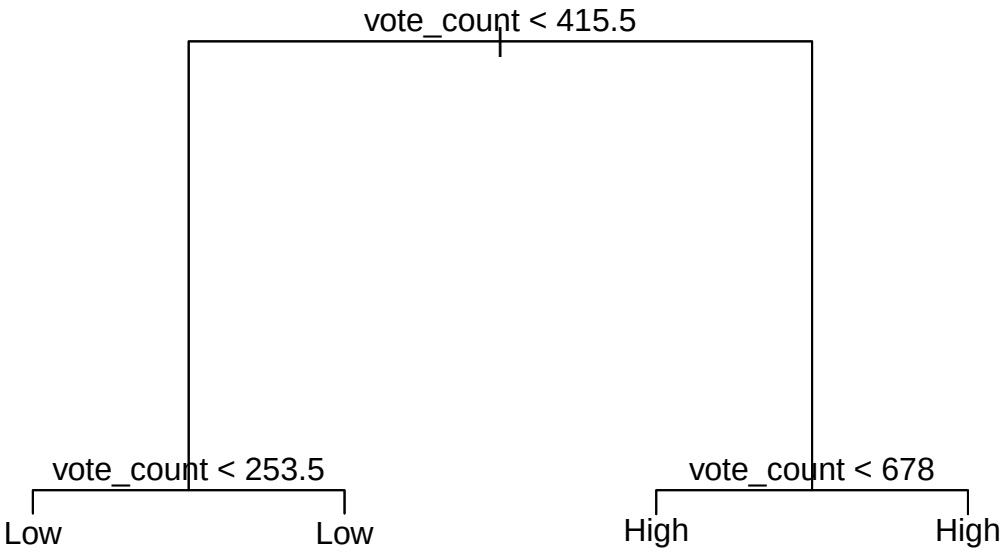



```r
library(stringr)

options(scipen = 999)

df_clean$popularity_cat <- ifelse(
  df_clean$popularity > median(df_clean$popularity),
  "High", "Low"
)
df_clean$popularity_cat <- as.factor(df_clean$popularity_cat)

df_tree <- subset(df_clean,
  select = c(popularity_cat, revenue, vote_average, vote_count, budget,num_genres, num_companies, num_la
)
```

```r
df_tree <- na.omit(df_tree)

library(tree)

set.seed(1)
train_index <- sample(1:nrow(df_tree), 0.8 * nrow(df_tree))

train_tree <- df_tree[train_index, ]
test_tree <- df_tree[-train_index, ]

tree.fit <- tree(
  popularity_cat ~ .,
  data = train_tree
)

summary(tree.fit)
```

```
##
## Classification tree:
## tree(formula = popularity_cat ~ ., data = train_tree)
## Variables actually used in tree construction:
## [1] "vote_count"
## Number of terminal nodes:  4
## Residual mean deviance:  0.4383 = 1130 / 2579
## Misclassification error rate: 0.07666 = 198 / 2583
```

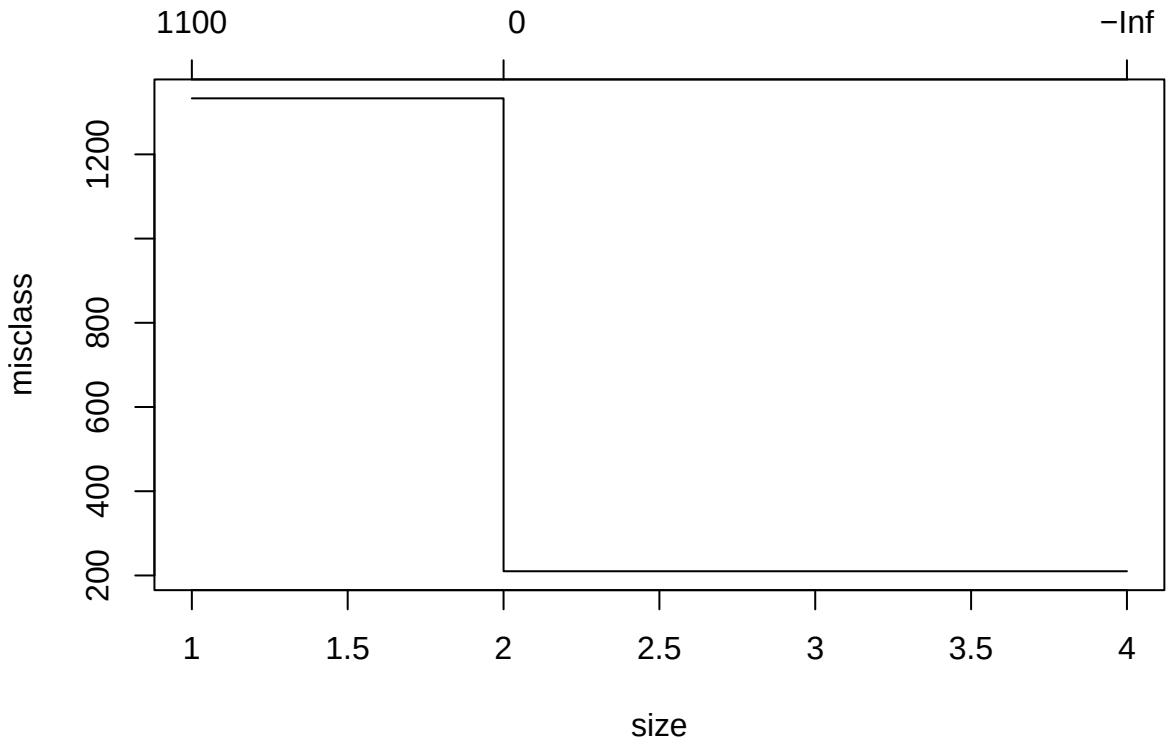
```r
plot(tree.fit)
text(tree.fit, pretty = 0)
```



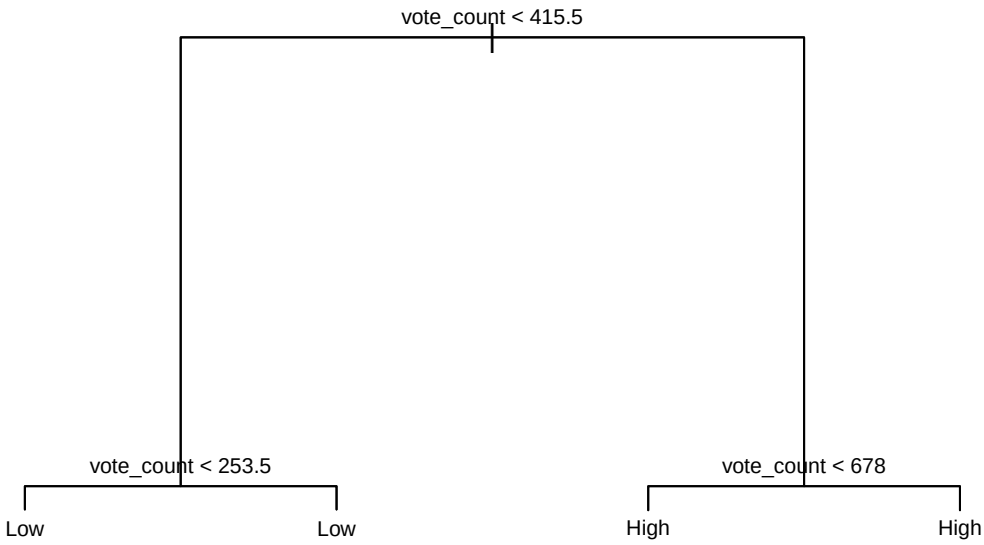

```
cv.tree.fit = cv.tree(tree.fit, FUN = prune.misclass)
summary(cv.tree.fit)
```

```
##        Length Class  Mode
## size   3      -none- numeric
## dev    3      -none- numeric
## k      3      -none- numeric
## method 1      -none- character
```

```
plot(cv.tree.fit)
```




```
best.size <- cv.tree.fit$size[which.min(cv.tree.fit$dev)]
pruned.tree = prune.misclass(tree.fit, best = best.size)
plot(pruned.tree)
text(pruned.tree, pretty = 0, cex = 0.7)
```




```
pruned.pred <- predict(pruned.tree, newdata = test_tree, type = "class")
acc = mean(pruned.pred == test_tree$popularity_cat)
acc
```

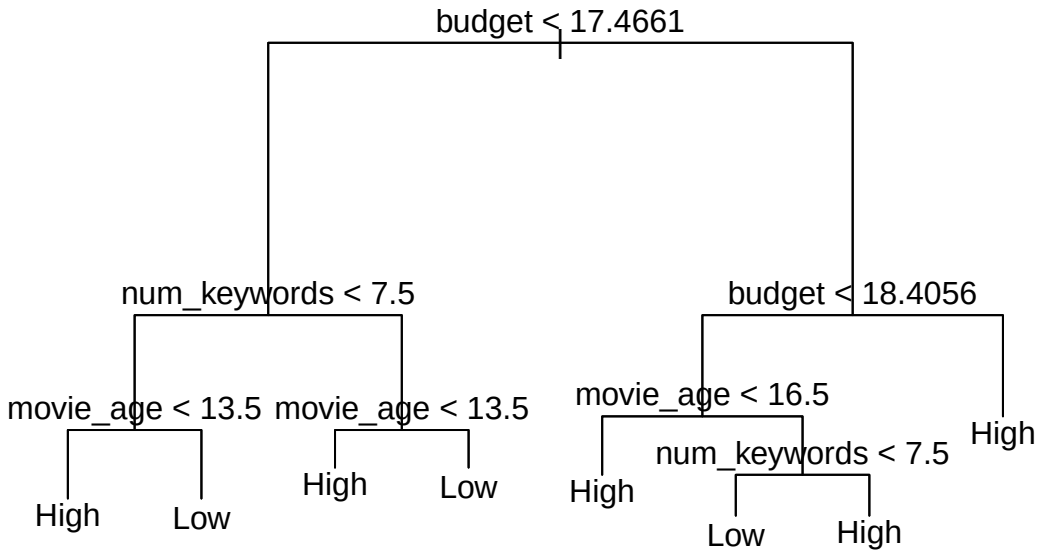
```
## [1] 0.9349845
```

```
df_clean$year <- as.numeric(df_clean$year)
```

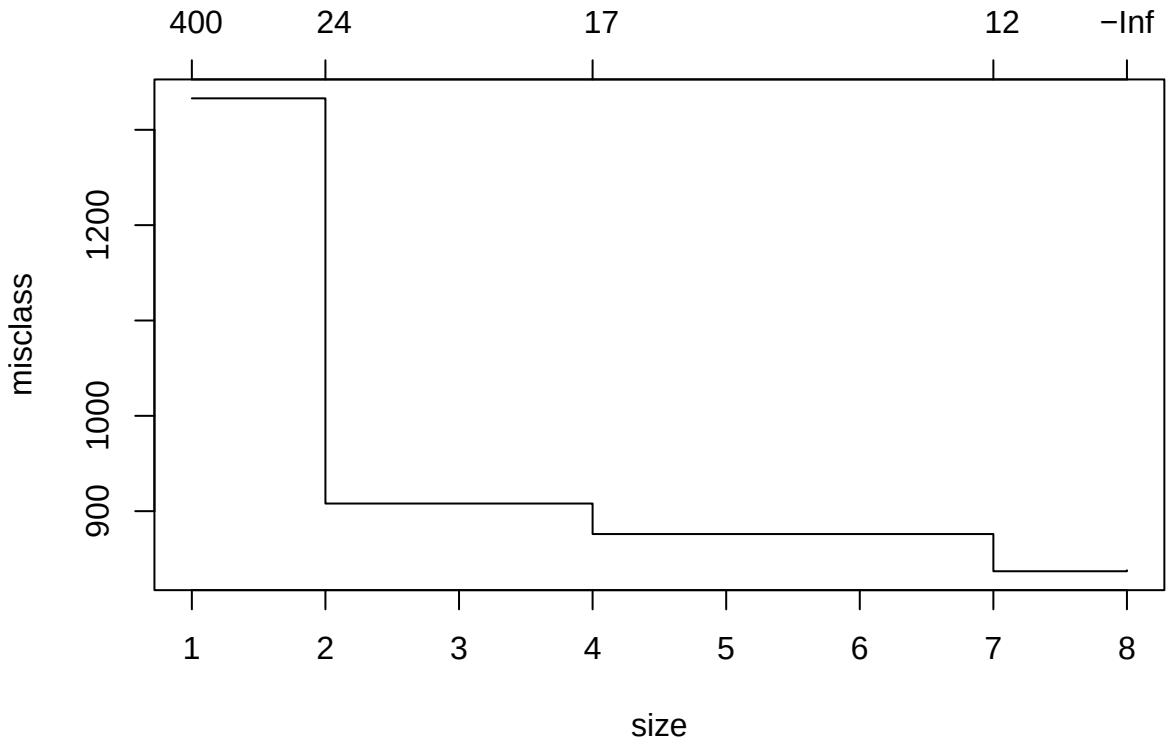
```
df_tree2 <- subset(df_clean,
  select = c(popularity_cat, budget, num_genres, num_companies,
             num_languages, num_keywords, movie_age)
)

df_tree2 <- na.omit(df_tree2)

set.seed(1)
train_index <- sample(1:nrow(df_tree2), 0.8 * nrow(df_tree2))

train_tree <- df_tree2[train_index, ]
test_tree <- df_tree2[-train_index, ]

tree.fit <- tree(
  popularity_cat ~ .,
  data = train_tree
)

summary(tree.fit)
```

```
##
## Classification tree:
## tree(formula = popularity_cat ~ ., data = train_tree)
## Variables actually used in tree construction:
## [1] "budget"       "num_keywords" "movie_age"
## Number of terminal nodes:  8
## Residual mean deviance:  1.118 = 2880 / 2575
## Misclassification error rate: 0.2997 = 774 / 2583
```

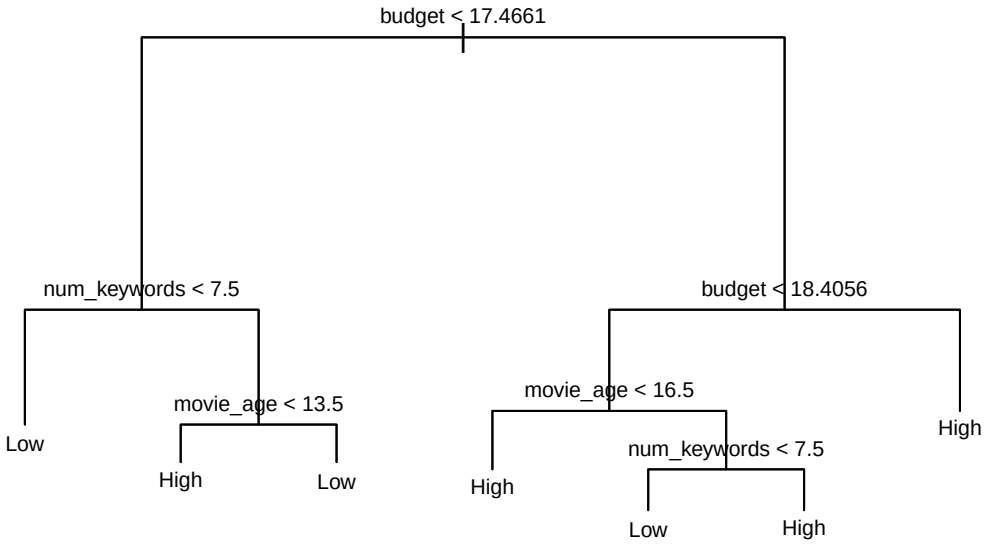
```
plot(tree.fit)
text(tree.fit, pretty = 0)
```




```
cv.tree.fit <- cv.tree(tree.fit, FUN = prune.misclass)
plot(cv.tree.fit)
```




```
best.size <- cv.tree.fit$size[which.min(cv.tree.fit$dev)]

pruned.tree <- prune.misclass(tree.fit, best = best.size)

plot(pruned.tree)
text(pruned.tree, pretty = 0, cex = 0.7)
```




```
pruned.pred <- predict(pruned.tree, newdata = test_tree, type = "class")
acc = mean(pruned.pred == test_tree$popularity_cat)
acc
```

```
## [1] 0.6873065
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.3.3
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
random.forest.fit = randomForest(popularity_cat ~ ., data = train_tree, ntree = 200)
random.forest.fit
```

```
##
## Call:
##  randomForest(formula = popularity_cat ~ ., data = train_tree,      ntree = 200)
##                Type of random forest: classification
##                      Number of trees: 200
## No. of variables tried at each split: 2
##
##         OOB estimate of  error rate: 29.38%
## Confusion matrix:
##      High Low class.error
## High  879 401   0.3132812
## Low   358 945   0.2747506
```

```
random.forest.pred = predict(random.forest.fit, newdata = test_tree, type = "class")
acc = mean(random.forest.pred == test_tree$popularity_cat)
importance(random.forest.fit)
```

```
##               MeanDecreaseGini
## budget                380.95016
## num_genres             93.94575
## num_companies         138.59046
## num_languages          66.81555
## num_keywords          268.89476
## movie_age             285.13867
```

```
print('accuarcy')
```

```
## [1] "accuarcy"
```

```
acc
```

```
## [1] 0.7151703
```

```
test_err = 1 - acc
print('error')
```

```
## [1] "error"
```

```
test_err
```

```
## [1] 0.2848297
```